



TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

TÉCNICA DE PROGRAMAÇÃO I



TÉCNICA DE PROGRAMAÇÃO I
PROFº LUIZ CLÁUDIO



Interface gráfica com Usuário – GUI(Graphical User Interface)

É onde os resultados são apresentados em modo gráfico. Essa interface é formada através de componentes GUI, conhecidos por **controles** ou **widgets**. Esses componentes são objetos que fazem a interação com usuário por teclado, mouse ou outros dispositivos que venham a servir para entrada de dados.

NETBEANS - GUI



Os componentes GUI Swing estão dentro do pacote **javax.swing** que são utilizados para construir as interfaces gráficas. Alguns componentes não são do tipo GUI Swing e sim componentes **AWT**. Antes de existir o GUI Swing, o Java tinha componentes **AWT** (Abstract Windows Toolkit) que faz parte do pacote **javax.awt**.

Controles Swing

Label Rótulo

OK Button

ON Botão Alternar

Caixa de seleção

Caixa de combinação

Lista

AWT

A Rótulo

ab Campo de texto

Caixa de seleção

Lista

Painel de rolagem

Canvas

Menu pop-up

OK Button

Área de texto

Opções

Barra de rolagem

Painel

Barra de menu

NETBEANS – COMPONENTES UTILIZADOS



Abaixo são mostrados alguns dos componentes mais usados:

❑ **JLabel** - Exibe texto não editável (LABEL)

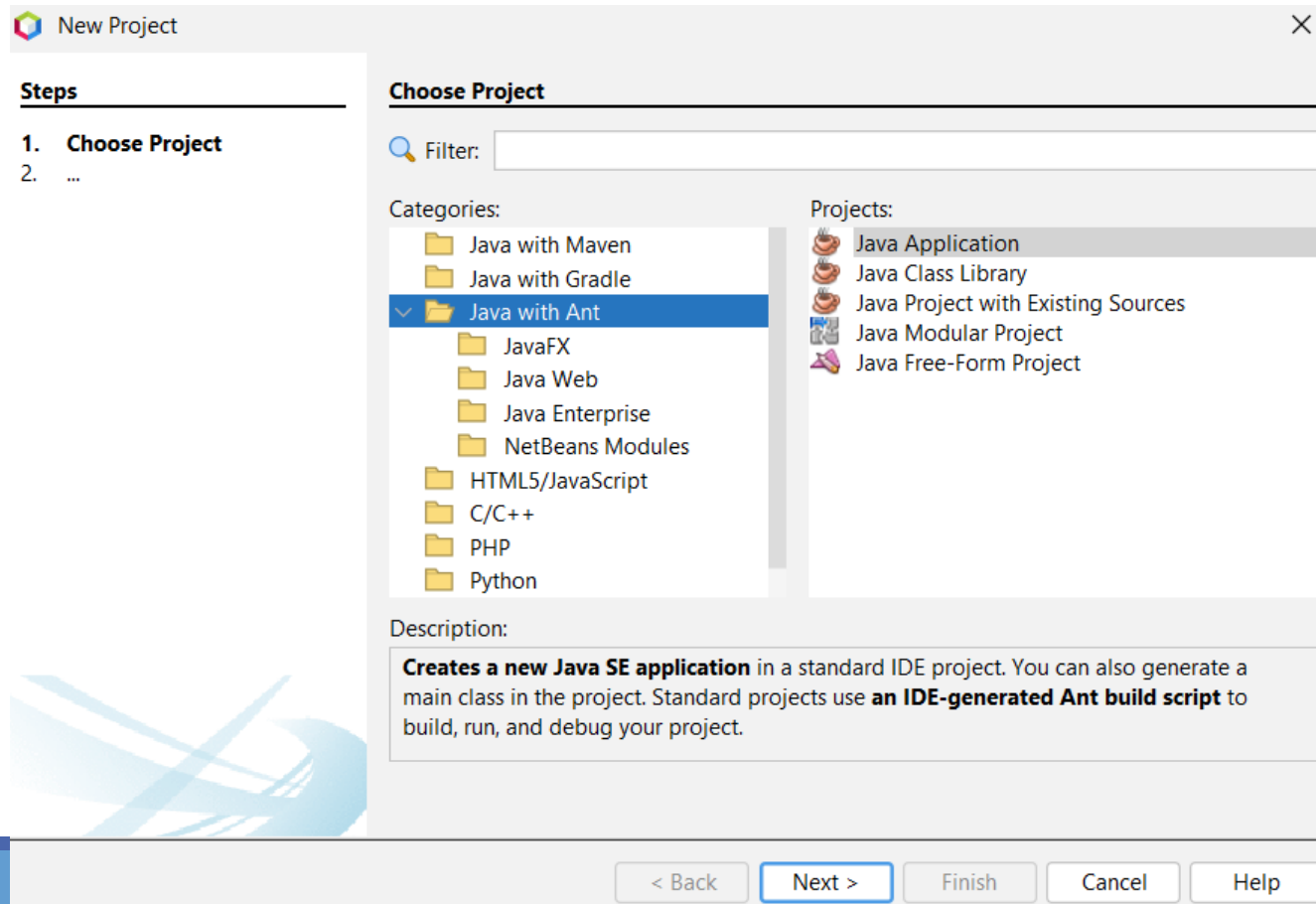
❑ **TextField** – Caixa de Texto

❑ **Button** – Botão - Libera um evento quando o usuário clicar nele com o mouse.

NETBEANS – COMPONENTES UTILIZADOS



Para criar um Projeto com formulário, crie um projeto como Ant , Java with Ant , Java Application



NETBEANS – COMPONENTES UTILIZADOS



Desmarque a caixa Create Main Class, para não criar a classe Principal

New Java Application

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Project Name:

Project Location:

Project Folder:

☐ Use Dedicated Folder for Storing Libraries

Libraries Folder:

Different users and projects can share the same compilation libraries (see Help for details).

☐ Create Main Class

< Back Next > **Finish** Cancel Help

Criar os pacotes Model, View, Control,
sistema MVC

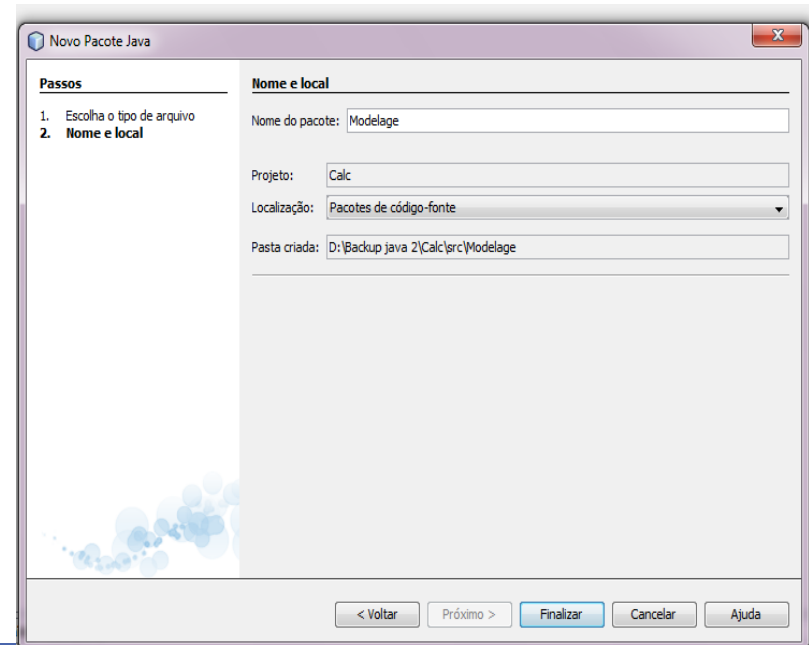
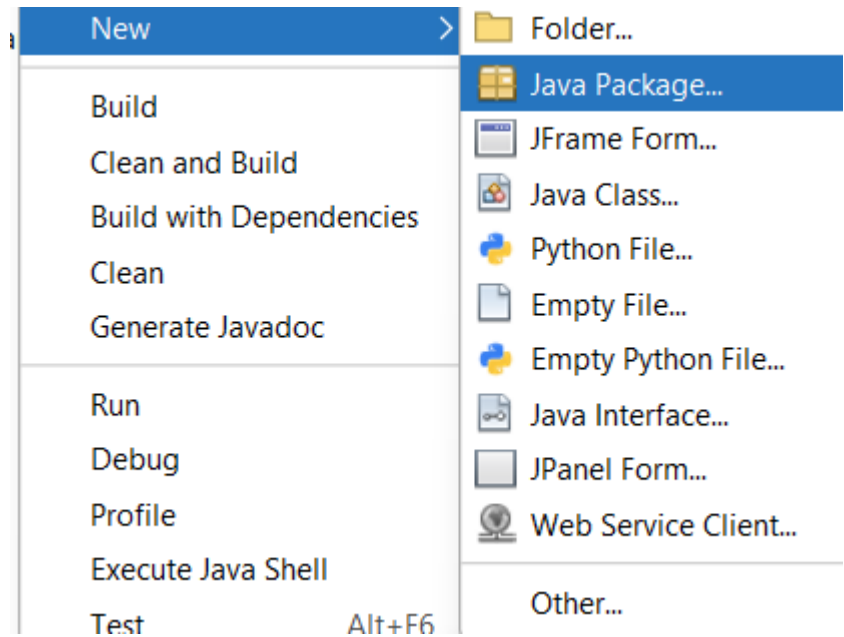
Criar as classes no pacote Model

Criar o formulário no pacote View

Criar a classe de conexao no pacote Control

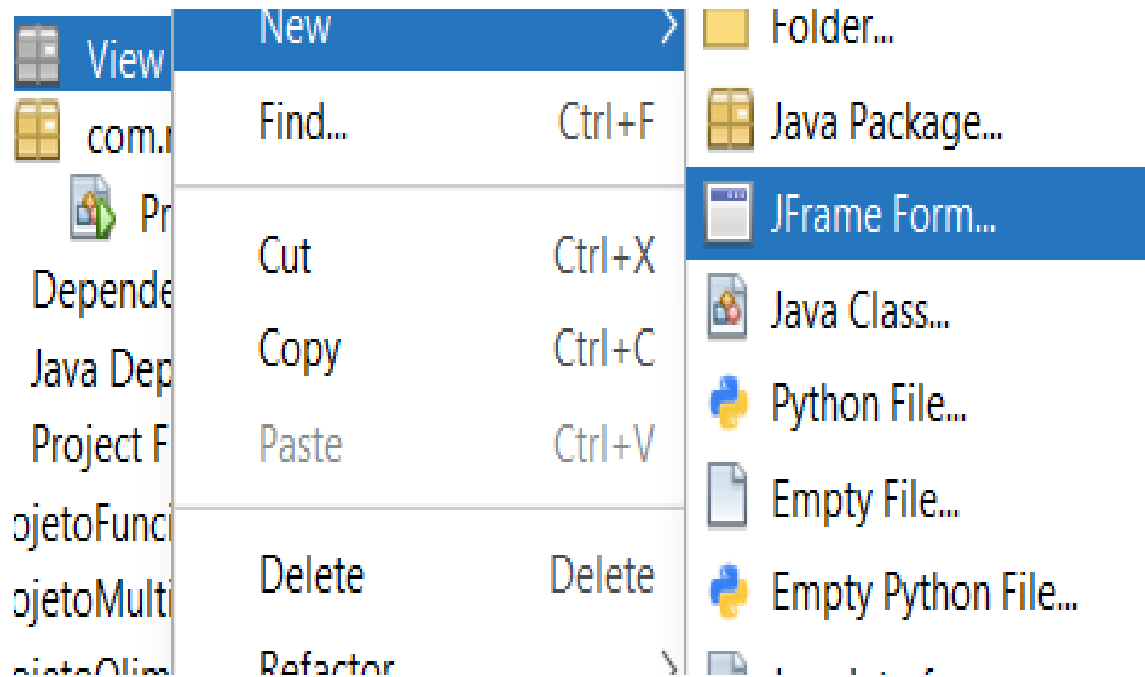
Para criar pacotes faça os passos a seguir:

1. Clicar com o botão direito em cima do nome do projeto
2. Depois New-> Java Package
3. Em seguida o nome do pacote



PARA CRIAR UM FORMULÁRIO JFRAME

1. CLICAR COM O BOTÃO DIREITO NO NOME DO PACOTE VIEW



Steps

1. Choose File Type
2. **Name and Location**

Name and LocationClass Name: Project: Location: Package: Created File:

Superclass:

Interfaces:

< Back

Next >

Finish

Cancel

Help

Coloque o nome do Formulário JFrame , sempre Inicie com nome Frm, neste caso FrmCliente

Construa o Formulário com os componentes e mude o nome das variáveis

The image shows a Java Swing application window titled "Cadastrar Cliente". It contains four text input fields labeled "Codigo", "Nome", "Telefone", and "Email", and a "Cadastrar" button. Below the inputs is a table with 4 rows and 3 columns: "Codigo", "Nome", and "Email". The table contains the following data:

Codigo	Nome	Email
1	Andreia	andreia@hotmail.com
2	Jose	abndmd@hotmail
3	Simone	simone@hotmail
4	Gilda	gilda@hotmail

Annotations and IDE settings are provided for each component:

- Componente JTextField /campo de texto**
VariableName: txtCodigo
- Componente JTextField /campo de texto**
VariableName : txtNome
- Componente JTextField /campo de texto**
VariableName : txtEmail
- Componente JButton**
VariableName : btnCadastrar
- Componente Jtable / Tabela**
VariableName : jtblClientes

IDE settings for the table component:

Propriedades	Vinculação	Eventos	Código
background		☐ [255,255,255]	...
border		(Sem Borda)	...
font		Tahoma 11 Simples	...
foreground		[0,0,0]	...
model		[Modelo de Tabela]	...

Componente Jtable / Tabela
VariableName : jtblClientes

Para adicionar as colunas na tabela altere a propriedade **model** e defina os títulos das colunas

Crie a tabela no BANCO DE DADOS

No WorkBench ou HeidiSQL, crie um database com nome banco_java, e crie a tabela Cliente

```
create database banco_java;
```

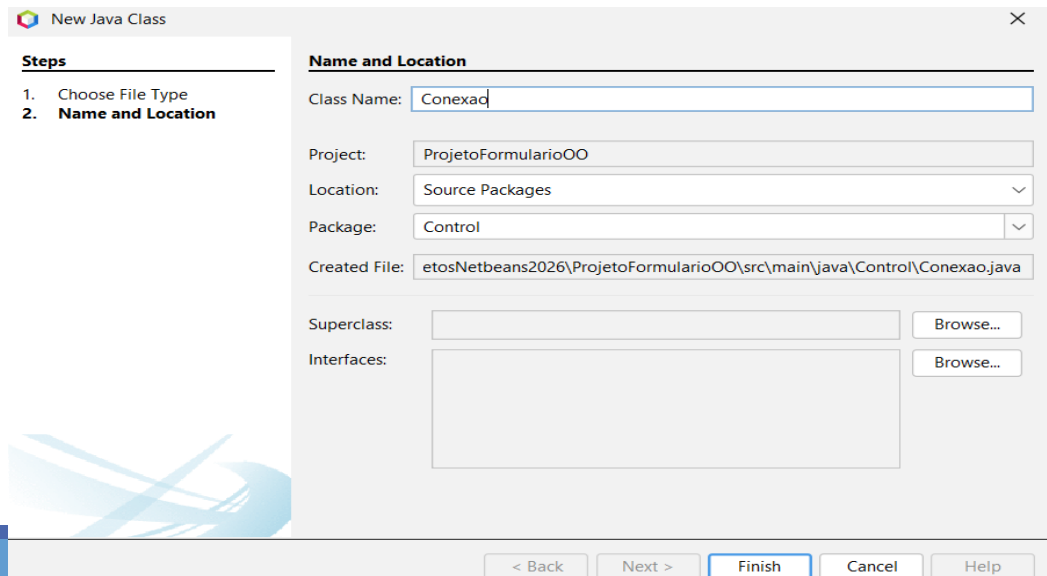
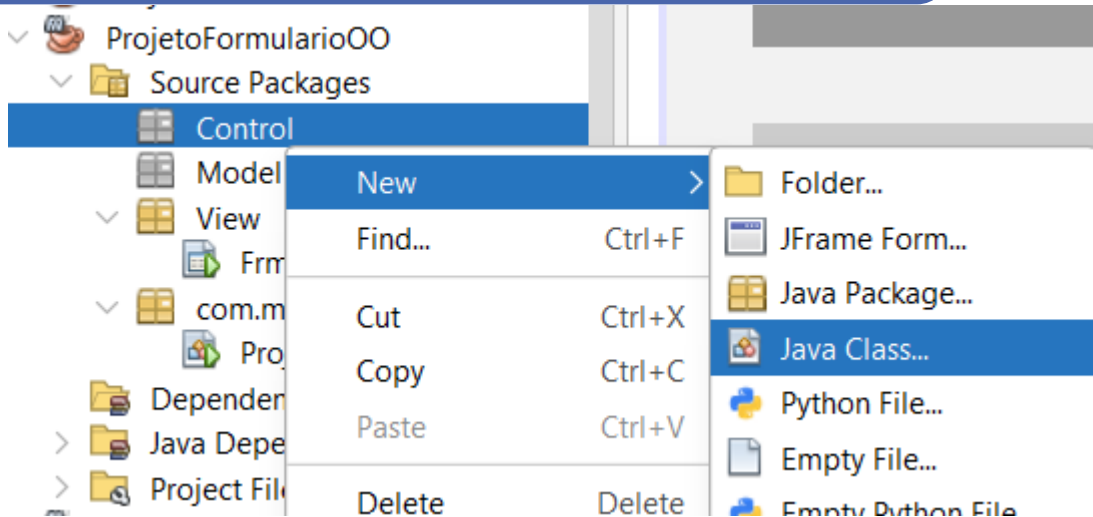
```
use banco_java;
```

```
create table cliente(  
id int auto_increment primary key,  
codigo int,  
nome varchar(70),  
email varchar(120),  
telefone varchar(18));
```

Crie classe Conexao-pacote Control

PARA CRIAR UMA
CLASSE DE
CONEXAO

2. CLICAR COM O
BOTÃO DIREITO NO
NOME DO PACOTE
CONTROL



Crie classe Conexao-pacote Control

```
package Controle;
```

```
import java.sql.*;  
import javax.swing.JOptionPane;
```

```
public class Conexao {
```

```
    final private String driver = "com.mysql.jdbc.Driver";
```

```
    final private String url= "jdbc:mysql://127.0.0.1/banco";
```

```
    final private String usuario="root";
```

```
    final private String senha="";
```

```
    private Connection conexao;// objeto que faz conexao com o banco
```

```
    public Statement statement;// objeto que abre caminho até o banco, cria a autoestrada.
```

```
    public ResultSet resultSet;// objeto que armazena os comandos sql
```

```
    public boolean conecta() {
```

```
        boolean result = true;
```

```
        try {
```

```
            Class.forName(driver);
```

```
            conexao = DriverManager.getConnection(url,usuario,senha);
```

```
            //JOptionPane.showMessageDialog(null,"Conectou com o Banco de Dados");
```

```
        } catch(ClassNotFoundException Driver){
```

```
            JOptionPane.showMessageDialog(null,"Driver nao localizado: "+Driver);
```

```
            result = false;
```

```
        }catch(SQLException Fonte) {
```

```
            JOptionPane.showMessageDialog(null,"Erro na conexão com a fonte de dados: "+Fonte);
```

```
            result = false;
```

```
        }
```

```
        return result;
```

Nome do banco de dados

```
final private String driver = "com.mysql.jdbc.Driver";  
final private String url= "jdbc:mysql://127.0.0.1/banco";
```

Endereço do mysql

Pode usar alguns casos localhost

Classe Conexao

```
public void desconecta () {
    boolean result = true;
    try
    {
        conexao.close();
        //JOptionPane.showMessageDialog(null, "Banco fechado");
    }
    catch(SQLException fecha)
    {
        JOptionPane.showMessageDialog(null, "Não foi possível fechar o banco de dados" + fecha);
        result = false;
    }
}

public void executeSQL(String sql) {
//chamada do metodo conecta para abrir a conexão com o db
    conecta();
    try{

        statement = conexao.createStatement();

        statement.execute(sql);
        //desconecta();
    }catch(SQLException sqle){
        JOptionPane.showMessageDialog(null, "Driver não encontrado!" + sqle.getMessage());
    }
}
```

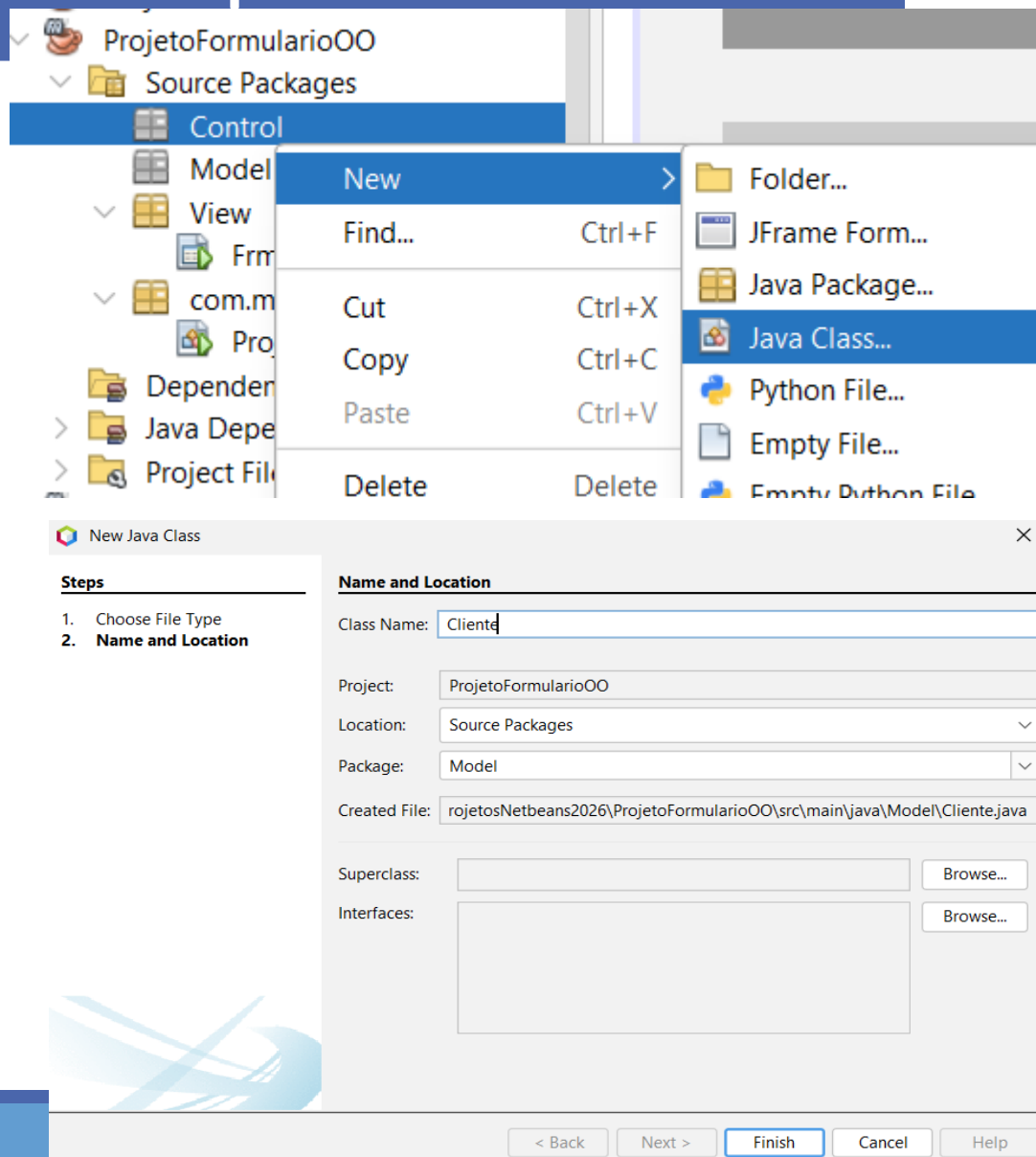
Classe Conexao

```
public ResultSet RetornarResultset(String sql){  
    ResultSet resultSet = null;  
    conecta();  
    try{  
        statement = conexao.createStatement();  
        resultSet = statement.executeQuery(sql);  
        resultSet.next();  
    }catch (Exception e){  
        JOptionPane.showMessageDialog(null, "Erro ao retornar resultSet"+e.getMessage());  
    }  
    return resultSet;  
}  
}
```

Crie classe Cliente-pacote Model

PARA CRIAR UMA
CLASSE DE
CLIENTES

3. CLICAR COM O
BOTÃO DIREITO NO
NOME DO PACOTE
MODEL



Criar a classe Cliente no pacote de Model

```
package Model;

public class Cliente {
    private int codigo;
    private String nome;
    private String telefone;
    private String email;

    public int getCodigo() {
        return codigo;
    }

    public void setCodigo(int codigo) {
        this.codigo = codigo;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getTelefone() {
        return telefone;
    }

    public void setTelefone(String telefone) {
        this.telefone = telefone;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Cliente

-codigo: int
-nome: String
-telefone: String
-email: String

+ cadastrar()
+ consultar()

Classe Cliente- Crie o método cadastrar

```
public class Cliente {  
    INSTANCIAR NA CLASSE CLIENTE A CLASSE DE CONEXÃO  
    Conexao concliente = new Conexao();  
  
    public void cadastrar() {  
  
        String sql= "Insert into cliente(codigo,nome,email,telefone)values "+  
        "("+getCodigo()+", '"+getNome()+"', '"+ getEmail()+"', '"+getTelefone()+"')";  
        concliente.executeUpdate(sql);  
        JOptionPane.showMessageDialog(null, "Registrado c sucesso");  
    }  
}
```

Classe Cliente- Crie o método consultar

```
public ResultSet consultar() {  
    ResultSet tabela;  
    tabela = null;  
  
    String sql= "Select * from cliente";  
    tabela = concliente.RetornarResultset(sql);  
    return tabela;  
}
```

Código do formulario

The image shows a Java Swing IDE with a form titled 'Cadastrar Cliente'. The form has fields for 'Nome', 'Telefone', and 'Email', a 'Cadastrar' button, a date/time picker, and a table of clients. Annotations point to various components and their variable names.

Componente JTextField /campo de texto
Nome da variável: txtCodigo

Componente JFormattedField Nome da variável: txtTelefone
Para Adicionar um mascara telefone **Formatter Factory**, defina em mask a mascara

Componente JTextField /campo de texto
Nome da variável: txtNome

Componente JTextField /campo de texto
Nome da variável: txtEmail

Componente JButton
Nome da variável: btnCadastrar

Componente Jtabela / Tabela
Nome da variável: jtblClientes

Para adicionar as colunas na tabela altere a propriedade **model** e defina os títulos das colunas

txtCodigo [JTextField] - Propriedades

Propriedades	Vinculação
Nome da Variável	txtCodigo
Modificadores de Variáveis	private

Propriedades Vinculação Eventos Código

Propriedades	Vinculação	Eventos	Código
background		☐ [255,255,255]	...
border		(Sem Borda)	...
font		Tahoma 11 Simples	...
foreground		[0,0,0]	...
model		[Modelo de Tabela]	...

Category Format

Category	Format
number	###-####
date	
time	
percent	
mask	

Format: (##)#####-####

Example

Value to format: 4343

Result: Invalid ch

Codigo	Nome	Email
1	Andreia	andreia@hotmail.com
2	Jose	abndmd@hotmail
3	Simone	simone@hotmail
4	Gilda	gilda@hotmail

Código do formulário – Instanciar classe Cliente

INSTANCIAR NO FORMULÁRIO A CLASSE DE CLIENTE

```
public class FCliente extends javax.swing.JFrame {
```

```
    public FCliente() {  
        initComponents();
```

```
    }
```

```
    Cliente cli = new Cliente();
```

Código do formulário – Botão Cadastrar

```
private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {  
cli.setCodigo(Integer.parseInt(txtCodigo.getText()));  
cli.setNome(txtNome.getText());  
cli.setEmail(txtEmail.getText());  
cli.setTelefone(txtTelefone.getText());  
cli.cadastrar();  
}
```

Colocar nome dos atributos , que recebe os valores das caixas de texto

`cli.setNomeatributo(nomeCaixaDeTexto.getText())`

```
ResultSet tabela;  
tabela = null;
```

Chama o método consultar da classe Cliente , que retorna os dados consultados no banco de dados

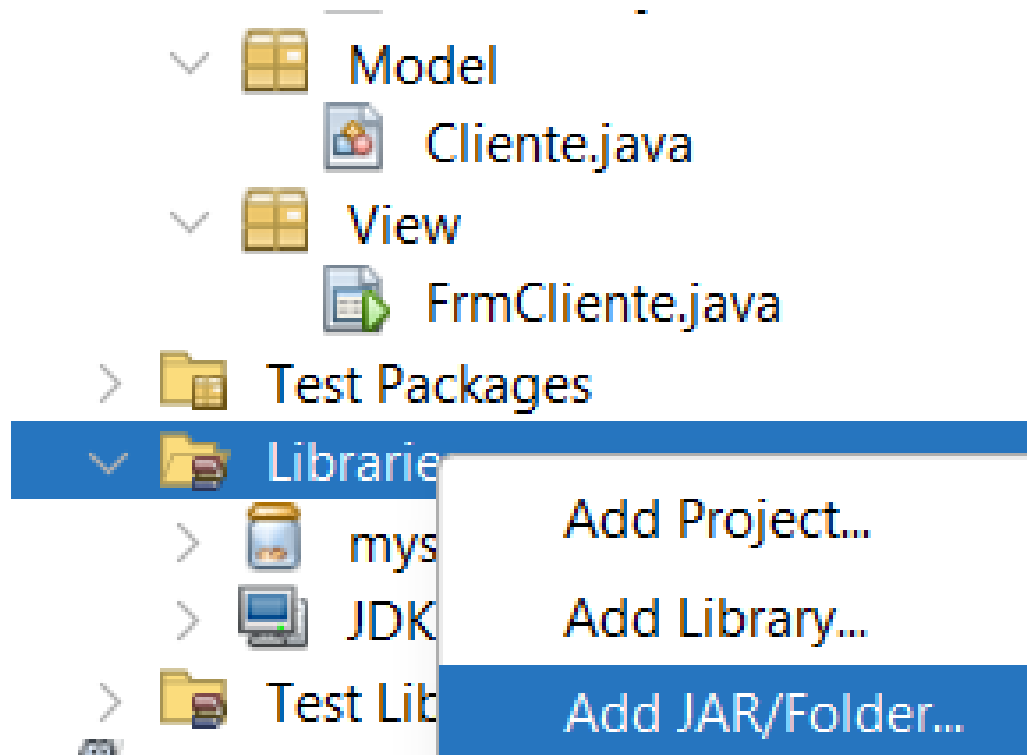
```
tabela = cli.consultar();  
DefaultTableModel modelo = (DefaultTableModel) jtbTabela.getModel();  
modelo.setNumRows(0);  
try  
{  
do{  
modelo.addRow(new String[]{tabela.getString(1), tabela.getString(2), tabela.getString(3), tabela.getString(4)});  
}  
while(tabela.next());  
}catch(SQLException erro)  
{  
JOptionPane.showMessageDialog(null, "Erro ao preencher tabela"+ erro) ;  
}  
}
```

Nome Da tabela utilizado no formulário JFrame

Sequencia dos dados que serão mostrados na tabela, de acordo [com banco de dados

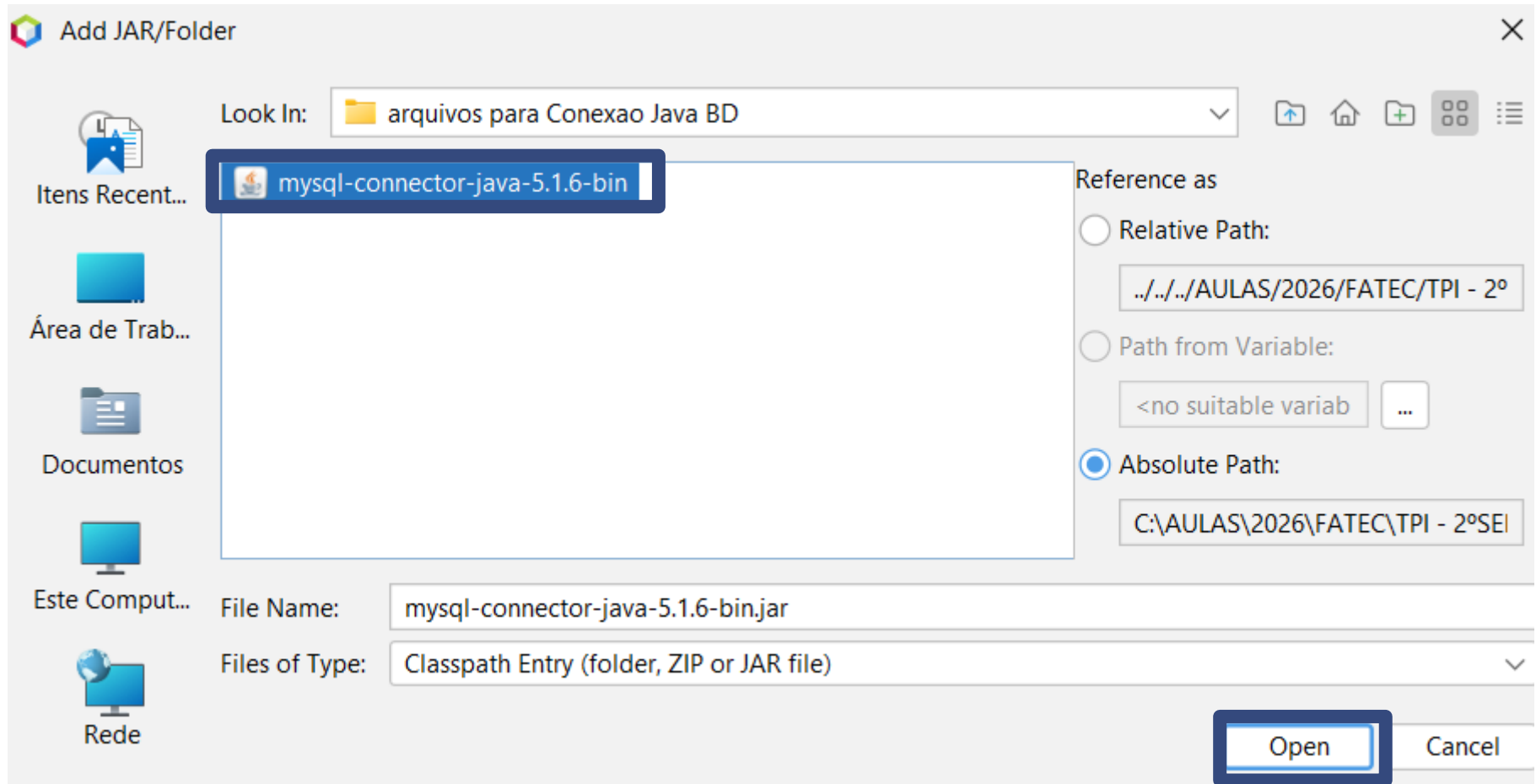
Incluir Conector –MySQL no Projeto

Clique com botão direito em Libraries -> Add JAR/Folder...



Incluir Conector -MySQL

Procure o conector do Java e clique em Open



Exercicio 1

1. Crie os pacotes de Model, Control e View.
2. Crie a classe Usuario em seu pacote designado, com os atributos nome, login, senha, email.
3. Crie o banco de dados chamado Usuario Com os seguintes campos nome, login, senha, email
4. Crie a classe de Conexão.
5. Crie o formulário frmUsuario.

The screenshot shows a Java Swing window titled "Cadastro Usuário". The window has a light gray background and a title bar with standard Windows controls. The main content area is titled "Cadastro de Usuário" in a bold, red, italicized font. Below the title, there are four labels with corresponding text input fields: "Digite o Nome:", "Digite o e-mail:", "Digite o login:", and "Digite a senha:". Each label is in a bold, italicized font. Below the input fields, there are three buttons: "Cadastrar" (orange), "Limpar" (pink), and "Sair" (orange). At the bottom of the window, there is a table with four columns: "Nome", "Login", "Email", and "Senha". The table has five rows, with the first row being the header and the subsequent four rows being empty data rows.

Nome	Login	Email	Senha

Usuario

-nome: String
-email: String
-login: String
-senha: String

+cadastrarUsuario()
+listarUsuario()

Exercicio 2

1. Crie os pacotes de Model, Control e View.
2. Crie a classe Produto em seu pacote designado, com os atributos codigo, nomeProduto e Descrição.
3. Crie o banco de dados chamado ConsultaProduto. Com os seguintes campos: codigo, nomeProduto e Descrição.
4. Crie a classe de Conexão.
5. Crie o formulário frmProduto.

Cadastro de Produto

Codigo

Nome do Produto

Descrição

Codigo	Nome do Produto	Descrição

Produto

-codigo: int
-nomeProduto: String
-descricao: String

+cadastrarProduto()
+listarProduto()

Exercicio 3

1. Crie os pacotes de Model, Control e View.
2. Crie a classe Venda em seu pacote designado, com os atributos codigo, nomeVendedor e NomeProduto.
3. Crie o banco de dados chamado ConsultaVenda. Com os seguintes campos: codigo, nomeVendedor e NomeProduto.
4. Crie a classe de Conexão.
5. Crie o formulário frmVenda.

Cadastro de Venda

Codigo

Nome do Vendedor

Nome do produto

Codigo	Nome do Vendedor	Nome do produto

Gravar Limpar sair

Venda

-codigo: int
-nomeVendedor: String
-nomeProduto: String

+cadastrarVenda()
+listarVenda()